



**SYMBIOSIS INSTITUTE OF TECHNOLOGY (SIT)**

Constituent of Symbiosis International (Deemed University), Pune

(Established under Section 3 of the UGC Act of 1956 vide notification number F-9-12/2001-U-3 of the Government of India)

Re-Accredited by NAAC with 'A++' Grade

**Subject: Programming with Java**

**ASSIGNMENT No :3**

**TITLE :**Develop a Java application demonstrating method overloading and the use of static variables, methods in class design.

### **Problem Statement**

Develop a Java application demonstrating method overloading and the use of static variables, methods in class design.

### **Learning Objectives**

To implement method overloading and use static variables and methods in Java programs.

### **Theory**

Method overloading allows multiple methods to have the same name with different parameter lists, enabling compile-time polymorphism. The static keyword is used for variables and methods that belong to the class instead of individual objects. Static members share common data among all objects and reduce memory usage. These concepts improve code flexibility, readability, and efficient resource management.

### **Outcome**

```
cl1.java • Project Settings
VS code Folder stuff > cl1.java
1  class Calculator {
2      static int calculationCount = 0;
3
4      int add(int a, int b) {
5          calculationCount++;
6          return a + b;
7      }
8
9      double add(double a, double b) {
10         calculationCount++;
11         return a + b;
12     }
13
14     int add(int a, int b, int c) {
15         calculationCount++;
16         return a + b + c;
17     }
18 }
19
20 public class CalculatorDemo {
21     public static void main(String[] args) {
22         Calculator calc = new Calculator();
23
24         System.out.println("Sum of 5 + 10 = " + calc.add(5, 10));
25         System.out.println("Sum of 3.5 + 2.2 = " + calc.add(3.5, 2.2));
26         System.out.println("Sum of 1 + 2 + 3 = " + calc.add(1, 2, 3));
27
28         System.out.println("Total Calculations Performed: " + Calculator.calculationCount);
29     }
30 }
```

Sum of 5 + 10 = 15

Sum of 3.5 + 2.2 = 5.7

Sum of 1 + 2 + 3 = 6

Total Calculations Performed: 3

```

VS code Folder stuff > J cl1.java
1 // ===== File: RestaurantBillingDemo.java =====
2 class RestaurantBilling {
3     static int totalOrders = 0;
4
5     double calculateBill(double amount) {
6         totalOrders++;
7         return amount;
8     }
9
10    double calculateBill(double amount, double serviceChargePercent) {
11        totalOrders++;
12        return amount + (amount * serviceChargePercent / 100);
13    }
14
15    double calculateBill(double amount, double deliveryCharge, boolean isDelivery) {
16        totalOrders++;
17        return amount + deliveryCharge;
18    }
19 }
20
21 public class RestaurantBillingDemo {
22     public static void main(String[] args) {
23         RestaurantBilling billing = new RestaurantBilling();
24
25         double takeAwayBill = billing.calculateBill(500.0);
26         double dineInBill = billing.calculateBill(500.0, 10.0);
27         double deliveryBill = billing.calculateBill(500.0, 40.0, true);
28
29         System.out.println("Take-Away Bill: Rs. " + takeAwayBill);
30         System.out.println("Dine-In Bill (with 10% service charge): Rs. " + dineInBill);
31         System.out.println("Delivery Bill (with Rs.40 delivery charge): Rs. " + deliveryBill);
32
33         System.out.println("Total Orders Processed: " + RestaurantBilling.totalOrders);
34     }
}

```

```

Take-Away Bill: Rs. 500.0
Dine-In Bill (with 10% service charge): Rs. 550.0
Delivery Bill (with Rs.40 delivery charge): Rs. 540.0
Total Orders Processed: 3

```

**Conclusion:** Both programs show how method overloading picks the right version based on parameters, while static variables (calculationCount, totalOrders) track shared data across all calls/objects of the class.

### Exercise

1. Develop a **Calculator program** using overloaded methods for addition of integers and decimals. Use a static variable to count calculations.

2. Develop a **Restaurant Billing Application** where overloaded methods calculate bills for dine-in, takeaway, and delivery orders, while static variables track total orders.